

LAB ASSIGNMENT 11.2

2303A52488

LAB OBJECTIVES

USED AI TO ASSIST IN DESIGNING AND IMPLEMENTING FUNDAMENTAL DATA STRUCTURES IN PYTHON. LEARNED HOW TO PROMPT AI FOR STRUCTURE CREATION, OPTIMIZATION, AND DOCUMENTATION. IMPROVED UNDERSTANDING OF STACKS, QUEUES, LINKED LISTS, BINARY SEARCH TREES, AND HASH TABLES.

TASK 1: STACK USING AI GUIDANCE

PROMPTED AI: "HELP ME IMPLEMENT A STACK CLASS IN PYTHON WITH PUSH, POP, PEEK, IS_EMPTY METHODS, ADD COMMENTS AND EXAMPLE USAGE."

Stack Implementation

```

class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        return None

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

```

OUTPUT:

```

$ python stack.py Stack
created.
Pushed 10
Pushed 20
Pushed 30
Top element: 30 Popped:
30
Top element: 20
Stack size: 2
Is empty? False
Popped all, now empty: True

```

Stack supports LIFO operations correctly with $O(1)$ time for push/pop.

TASK 2: QUEUE DESIGN

PROMPTED AI: "CREATE A QUEUE CLASS IN PYTHON USING LIST FOR ENQUEUE, DEQUEUE, FRONT, SIZE, FIFO ORDER, WITH DOCUMENTATION."

QUEUE IMPLEMENTATION

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        return None

    def front(self):
        if not self.is_empty():
            return self.items[0]
        return None

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)
```

OUTPUT:

```
$ python queue.py Queue
created.
Enqueued 1
Enqueued 2
Enqueued 3
Front: 1
Dequeued: 1
Front now: 2
Queue size: 2
```

Is empty? False

QUEUE FOLLOWS FIFO PRINCIPLE. NOTE: POP(0) IS $O(N)$, BETTER WITH COLLECTIONS.DEQUE IN PRODUCTION.

TASK 3: SINGLY LINKED LIST CONSTRUCTION

PROMPTED AI: "BUILD A SINGLY LINKED LIST IN PYTHON WITH NODE CLASS, INSERT AT END/BEGINNING, DISPLAY TRAVERSAL."

LINKED LIST IMPLEMENTATION

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node
```

```
WHILE LAST.NEXT:      LAST = LAST.NEXT      LAST.NEXT = NEW_NODE
```

```
DEF DISPLAY(SELF):      ELEMENTS = []      TEMP = SELF.HEAD      WHILE TEMP:  
    ELEMENTS.APPEND(STR(TEMP.DATA))      TEMP = TEMP.NEXT      PRINT(" -> ".JOIN(ELEMENTS))
```

OUTPUT:

```
$ PYTHON LINKEDLIST.PY  
INSERTED 10  
INSERTED 20  
INSERTED 30  
DISPLAY: 10 -> 20 -> 30
```

LINKED LIST BUILT WITH DYNAMIC NODES, INSERTION AT END $O(N)$, TRAVERSAL $O(N)$.

TASK 4: BINARY SEARCH TREE OPERATIONS

PROMPTED AI: "IMPLEMENT BST IN PYTHON WITH INSERT, IN-ORDER TRAVERSAL, NODE CLASS."

BST IMPLEMENTATION

```
CLASS BSTNODE:  DEF  
    __INIT__(SELF, DATA):  
        SELF.DATA = DATA  
    SELF.LEFT = NONE  
    SELF.RIGHT = NONE  
  
CLASS  
BINARYSEARCHTREE:  
    DEF __INIT__(SELF):  
        SELF.ROOT = NONE
```

```

DEF INSERT(SELF, DATA):
    IF NOT SELF.ROOT:
        SELF.ROOT = BSTNODE(DATA)

    return
current = self.root
while True:
    if data < current.data:
        if current.left:
            current = current.left
        else:
            current.left = BSTNode(data)
        break
    else:
        if current.right:
            current = current.right
        else:
            current.right = BSTNode(data)
        break

def inorder(self, node):
    if node:
        self.inorder(node.left)
    print(node.data, end=" ")
    self.inorder(node.right)

```

Output:

```
$ python bst.py
Inserted 50 Inserted
30 Inserted 70
Inserted 20
Inserted 40
In-order traversal: 20 30 40 50 70
```

BST maintains sorted order, insert $O(h)$ where h is height, in-order gives ascending list.

TASK 5: HASH TABLE IMPLEMENTATION

PROMPTED AI: "CREATE HASH TABLE IN PYTHON WITH CHAINING COLLISION HANDLING, INSERT, SEARCH, DELETE."

HASH TABLE IMPLEMENTATION

```

CLASS HASHTABLE:
    DEF __INIT__(SELF,
    SIZE=10):
        SELF.SIZE = SIZE
        SELF.TABLE = [[] FOR _ IN RANGE(SIZE)]

        DEF HASH_FUNCTION(SELF, KEY):
    RETURN HASH(KEY) % SELF.SIZE

        DEF INSERT(SELF, KEY, VALUE):
            INDEX =
    SELF.HASH_FUNCTION(KEY)    FOR
    PAIR IN SELF.TABLE[INDEX]:
            IF PAIR[^1_0] == KEY:
    PAIR[^1_1] = VALUE        RETURN
    SELF.TABLE[INDEX].APPEND([KEY,
    VALUE])

            DEF SEARCH(SELF, KEY):
                INDEX =
    SELF.HASH_FUNCTION(KEY)
    FOR PAIR IN SELF.TABLE[INDEX]:
    IF PAIR[^1_0] == KEY:
    RETURN PAIR[^1_1]    RETURN
    NONE

            DEF DELETE(SELF, KEY):
                INDEX = SELF.HASH_FUNCTION(KEY)
    FOR I, PAIR IN
    ENUMERATE(SELF.TABLE[INDEX]):
    IF PAIR[^1_0] == KEY:

    SELF.TABLE[INDEX].POP(I)
    RETURN TRUE    RETURN
    FALSE

```

OUTPUT:


```
$ PYTHON HASHTABLE.PY
HASH TABLE SIZE: 10
INSERTED 'APPLE': 5
INSERTED 'BANANA': 3
SEARCH 'APPLE': 5
SEARCH 'GRAPE': NONE
DELETED 'APPLE'
SEARCH 'APPLE': NONE
```